

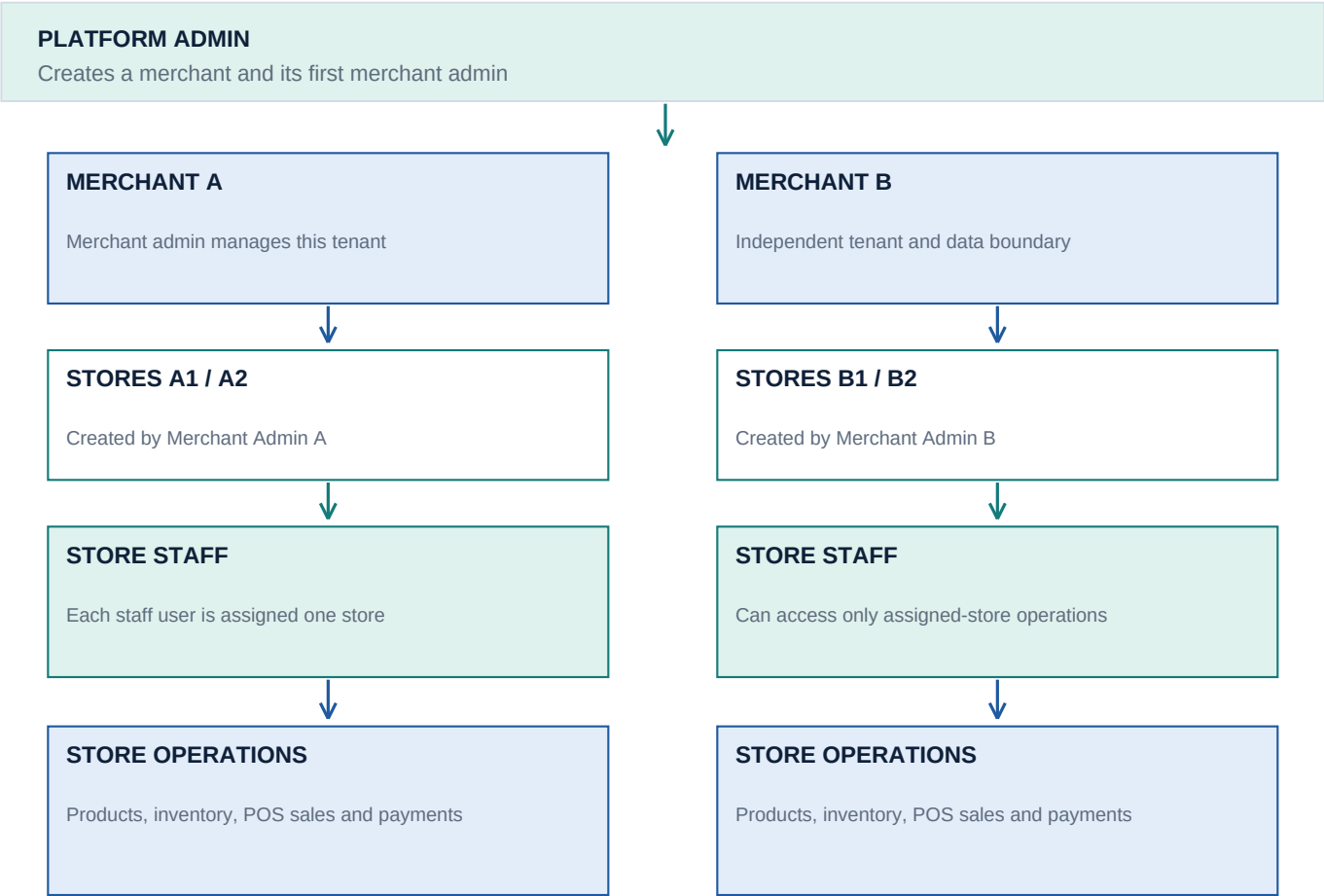
A point-of-sale system for multiple merchants

A concise guide to the application structure, boundaries, API and sale lifecycle.

01 / INTRODUCTION

Project overview

This NestJS API supports platform onboarding and independent merchant operations. A merchant owns stores; each store owns its product catalogue, inventory and sales. Merchant staff are restricted to their assigned store.



AT A GLANCE

API NestJS + TypeScript; JSON over HTTP; JWT bearer auth.	Data PostgreSQL via Sequelize; migrations define schema.	Runtime services Redis product cache; mock payment provider; Jest tests.
---	--	--

Seed examples are Zodio / Zodio Sector 86 and Urban Pantry / Urban Pantry Noida, with staff Rahul, Manish and John. The diagram reflects the implemented roles and tenant scope; it does not imply a separate service per merchant.

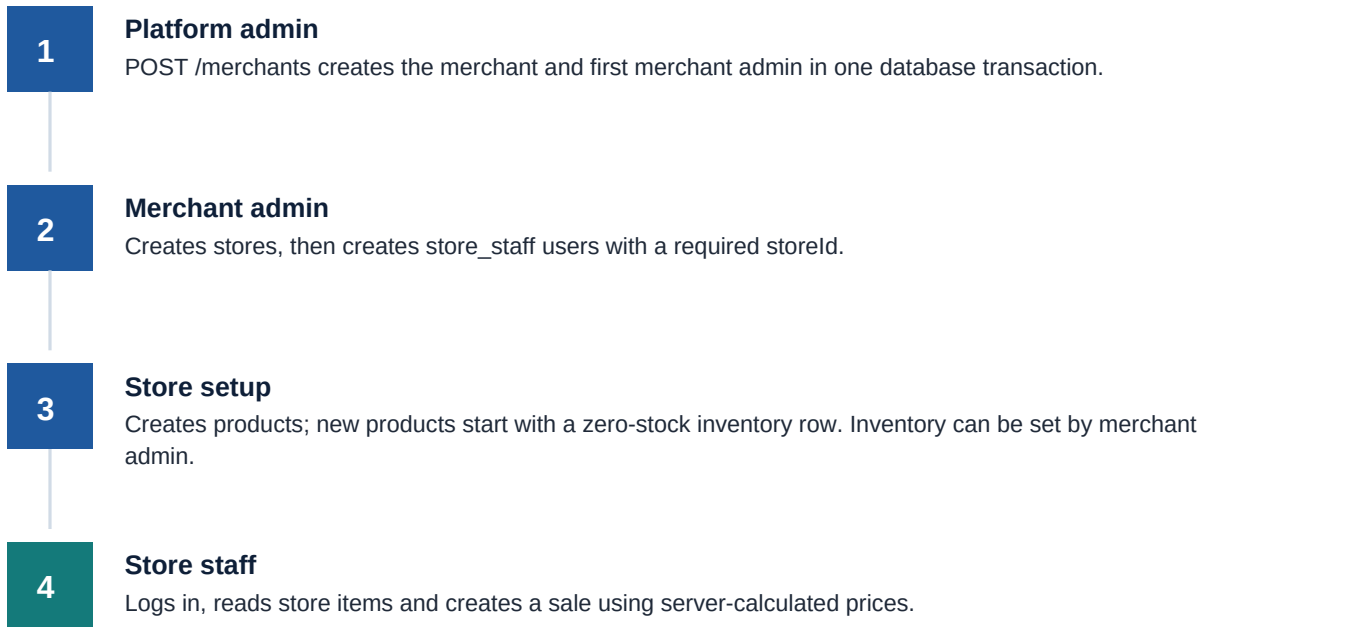
Roles, responsibilities and access

Authorization is enforced by JWT and role guards, then scoped to the merchant or store.

02 / FUNCTIONALITY

ROLE	CAN DO	CANNOT DO
platform_admin	Create a merchant and its first merchant admin.	Use tenant routes that are not explicitly platform-authorized.
merchant_admin	Manage own stores, staff, products, inventory and sales.	Read or modify another merchant tenant.
store_staff	Read store catalogue / inventory; create and read sales for assigned store.	Create users, adjust inventory, manage products or access another store.

How a merchant is provisioned

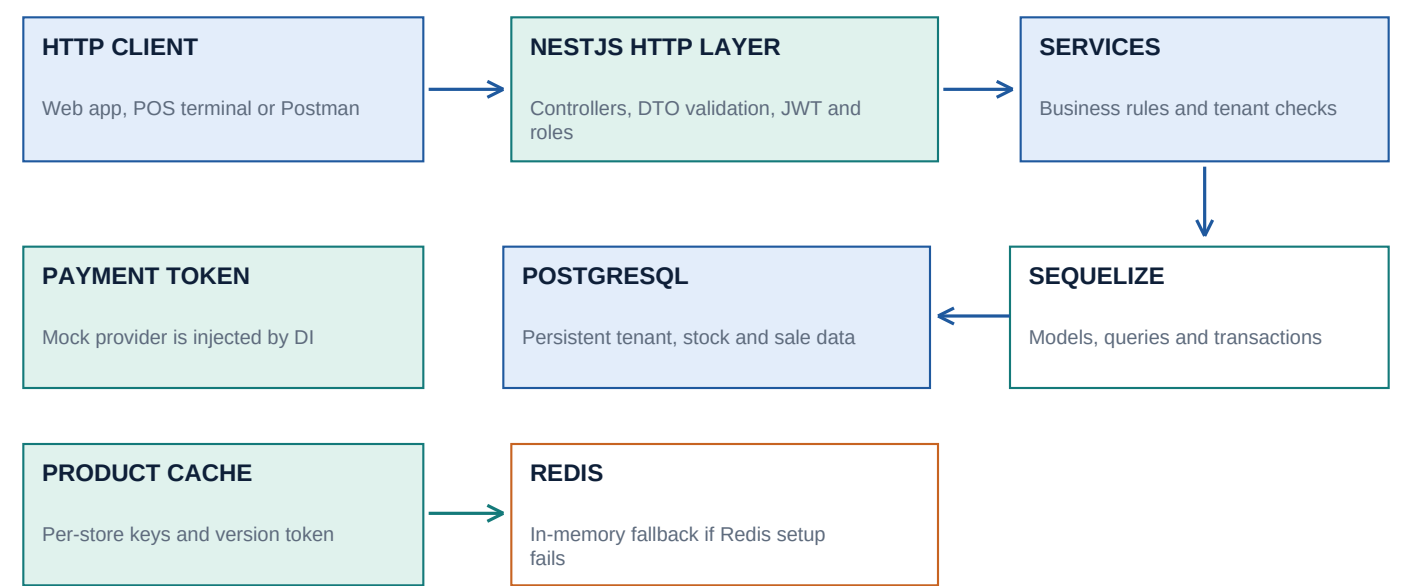


Tenant privacy behavior: out-of-scope merchant/store resources are returned as not found (404), rather than exposing their existence. A staff account is checked against its assigned store.

Application architecture

One NestJS application with feature modules and a shared PostgreSQL database.

03 / ARCHITECTURE



SalesService injects PAYMENT_PROVIDER -> mock provider.

Feature modules

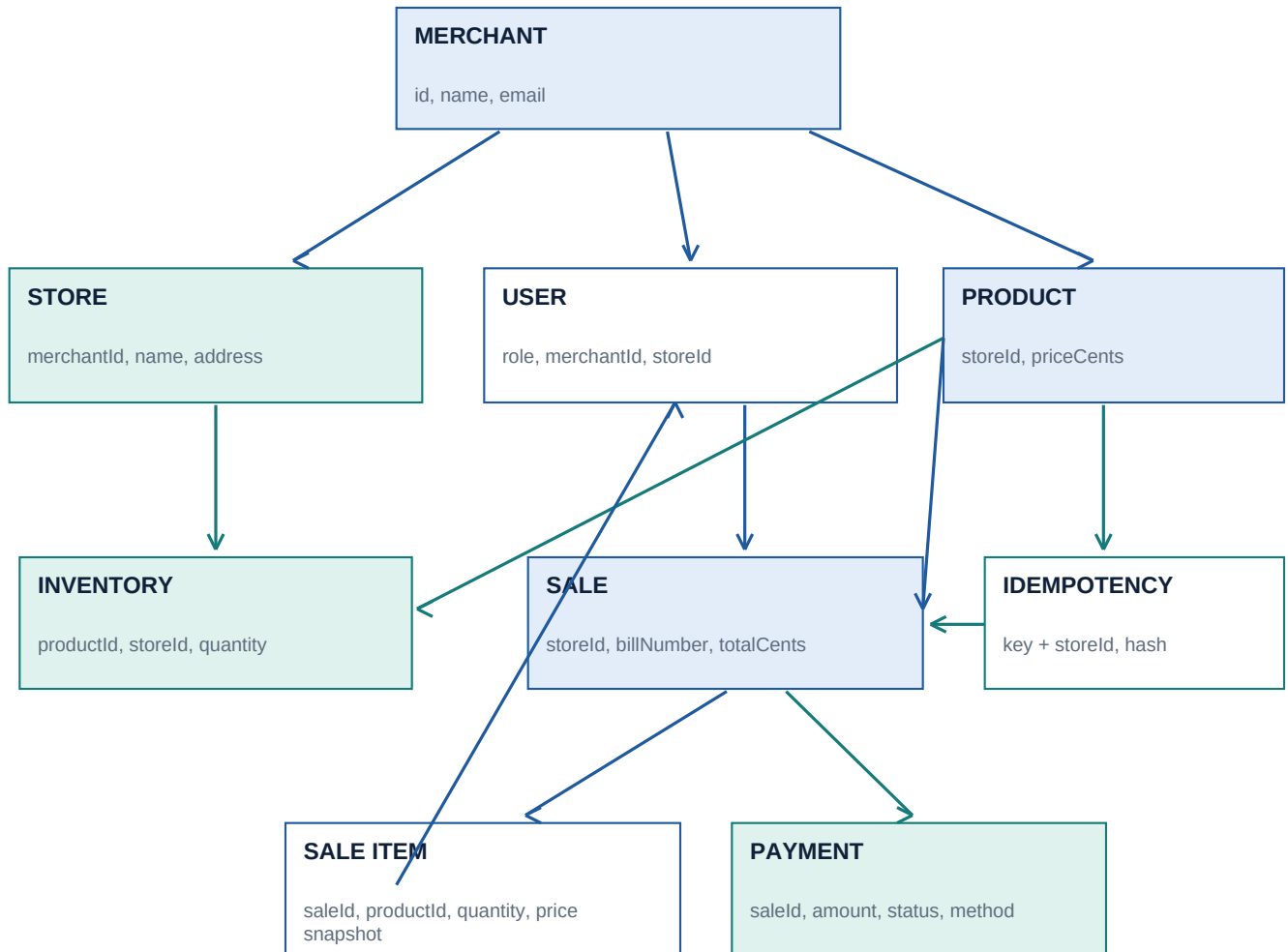
auth Login and JWT issuance	merchants Merchant onboarding and tenant reads
stores Store creation, listing and access checks	users Merchant admin creates store staff
products Catalogue and cached product listing	inventory Stock query and absolute adjustment
sales Sale transaction, idempotency and response	payments Provider contract and mock implementation
health / common Health checks, guards, filters and decorators	database Sequelize entities, migrations and demo seed

There is no separate repository class; services use Sequelize models directly. Global request validation rejects unknown DTO fields.

Core data model

UUID entities are connected through foreign keys; storeId is carried into store-owned records.

04 / DATA MODEL



IMPORTANT SCHEMA DECISIONS

Money

Product prices, sale totals, payment amounts and sale-item prices are stored as integer cents in BIGINT columns.

Stock safety

Inventory has a database CHECK constraint requiring quantity ≥ 0 and one inventory record per product.

Bill numbering

sales_bill_seq is global. PostgreSQL sequence values can have gaps after transaction rollback.

Idempotency

A unique (key, storeId) claim links a request hash to the completed sale response.

Sale creation: validated, locked, charged

The provider is called only after stock validation and sale rows have been prepared.

05 / POS SALE



Concurrency and retry behavior

For stock 5 and two concurrent sales of 3, the inventory row lock serializes the updates: one sale succeeds, the other receives insufficient stock, final stock is 2. PostgreSQL deadlock (40P01) and serialization (40001) errors may retry up to the existing limit only before payment is attempted. Errors after the provider call are rethrown. 23505 is not retried.

API reference: accounts and tenant setup

No global prefix. Protected routes use a bearer token in the Authorization header.

06 / HTTP API

ENDPOINT	ACCESS	BODY	PURPOSE
POST /auth/login	Public	email, password	JWT login
POST /merchants	platform_admin	name, email, adminName, adminEmail, adminPassword	Create merchant + admin
GET /merchants	Tenant users	-	Own merchant
GET /merchants/:id	Tenant users	-	Own merchant detail
POST /stores	merchant_admin	name, address?	Create own store
GET /stores	Tenant users	-	Visible stores
GET /stores/:id	Tenant users	-	Visible store detail
POST /users	merchant_admin	name, email, password, storeId	Create store staff
GET /users	merchant_admin	-	Own tenant users
GET /users/:id	merchant_admin	-	Own tenant user

Validation and authorization

The global ValidationPipe transforms DTO inputs, removes no extra fields silently, and rejects non-whitelisted properties. Login requires an email and password. Merchant onboarding creates the merchant and first merchant admin atomically. User creation always creates store_staff and requires a storeId within the admin merchant.

Health checks are public: GET /health, GET /health/database and GET /health/cache.

API reference: store operations

Route store IDs are checked against both the caller tenant and assigned staff store.

07 / STORE API

ENDPOINT	ACCESS	BODY / HEADER	PURPOSE
POST /stores/:storeId/products	merchant_admin	name, priceCents, description?, sku?	Create product
GET /stores/:storeId/products	Tenant users	-	List active products
GET /stores/:storeId/products/:id	Tenant users	-	Read active product
PUT /stores/:storeId/products/:id	merchant_admin	name?, priceCents?, description?, sku?	Update product
DELETE /stores/:storeId/products/:id	merchant_admin	-	Deactivate product
GET /stores/:storeId/inventory	Tenant users	-	List inventory
PUT /stores/:storeId/inventory/products/:productId	merchant_admin	quantity	Set stock
POST /stores/:storeId/sales	Tenant users	items, paymentMethod, notes?; Idempotency-Key header	Create sale
GET /stores/:storeId/sales	Tenant users	-	List sales
GET /stores/:storeId/sales/:id	Tenant users	-	Read sale

Cache and payment behavior

Idempotency

Send Idempotency-Key in the header. Same key and payload replays the sale; different payload returns 422. Claims are unique per store and created in the transaction.

Product cache

Active product lists with inventory are cached per store. TTL is 60 seconds; version changes invalidate after updates and committed sales. Cache timeout is 500 ms; Redis failure falls back to memory.

Payment mock

PAYMENT_FAIL_RATE=0 always succeeds; 1 always fails. Failed payment rolls back sale and stock data, but the external provider call cannot be rolled back or refunded.

Tenant users means merchant_admin or store_staff. Sale items contain productId and quantity; paymentMethod is cash, card or upi. Prices are never accepted from the client.

Running, testing and project notes

Requirements: Node.js + npm and PostgreSQL; Redis is optional.

08 / OPERATIONS

Quick start

```
git clone https://github.com/geekycodermanish/Multi-Tenant-POS.git
cd Multi-Tenant-POS && npm install
cp .env.example .env (set PLATFORM_ADMIN_EMAIL and PLATFORM_ADMIN_PASSWORD)
createdb swazei_pos
npm run migration:run && npm run seed
npm run start:dev
curl http://localhost:3000/health/database
```

Runtime configuration

VARIABLE	PURPOSE	EXAMPLE
PORT / NODE_ENV	Listen port / environment	3000 / development
DB_HOST / DB_PORT / DB_USERNAME / DB_PASSWORD / DB_NAME	PostgreSQL connection	localhost / 5432 / swazei_pos
JWT_SECRET / JWT_EXPIRES_IN	JWT signing / token lifetime	set a private secret / 8h
PLATFORM_ADMIN_EMAIL / PASSWORD	Default initial platform admin	set locally; password >= 8 chars
REDIS_HOST / REDIS_PORT / REDIS_TTL	Redis product cache	localhost / 6379 / 300
PAYMENT_FAIL_RATE	Mock payment failure chance	0 succeeds, 1 always fails

Code and test layout

src/modules/{auth,merchants,stores,users,products,inventory,sales,payments} contains the feature modules. src/common contains guards, decorators, exception filter and health checks. src/database contains entities, migrations and seed. Unit specs are colocated under src; integration suites cover auth, onboarding, tenant isolation, sales, inventory, idempotency, concurrency and regressions.

Useful commands: npm run test:unit, npm run test:int, npm run build. Integration setup requires a separate test database; read test/jest.global-setup.ts before running it because the setup recreates the public schema.

Implementation boundaries to know

- Payment runs while stock row locks are held; acceptable for this fast mock, not a production payment flow.
- A failed charge rolls back database sale and stock data, but cannot undo or refund the external provider call.
- Idempotency claims do not expire; bill numbers are global and sequence values may have gaps after rollback.
- Product-list cache uses per-store versions; list TTL is 60 seconds and cache calls time out after 500 ms.